

ESTRUTURA DE DADOS II - PROF. ALEXANDRE GOMES

Lista de Exercícios de Revisão do 1º Bimestre

Busca, Recursividade e Ordenação em JavaScript

Disciplina:	Estrutura de Dados
Curso:	Ciência da Computação / Sistemas de Informação / Engenharia de Software (UniFACEF)
Professor:	Mestre Alexandre Gomes
Período letivo:	1º Bimestre do 2º semestre de 2026
Linguagem:	JavaScript (Node.js, módulos .mjs)
Finalidade:	Revisão para a prova, uso livre, sem entrega

Instruções gerais

Esta lista retoma os conteúdos trabalhados no 1º bimestre: algoritmos de busca (sequencial e binária), recursividade e algoritmos de ordenação (bubble sort, selection sort, merge sort e quick sort), com implementações em JavaScript sobre vetores simples e vetores de objetos.

Esta é uma atividade de estudo, sem entrega. Resolva os exercícios do jeito que for mais útil para a sua revisão: no VS Code, com anotações à mão, no papel, como preferir. Sempre que um exercício pedir para "montar uma tabela", "explicar" ou "anexar" algo, isso é apenas uma sugestão de como organizar o raciocínio: guarde o resultado onde for melhor para você estudar, não é necessário formalizar nada.

- Use como ponto de partida os arquivos .mjs já trabalhados em sala e os conjuntos de dados da pasta data/ (vetor-nomes.mjs, nomes-desord.mjs, vetor-obj-nomes.mjs e motoristas-obj-desord.mjs).
- Cada exercício indica o arquivo-modelo em que ele se baseia. Reaproveite a estrutura e os comentários no mesmo estilo (condição de saída, condição de percurso etc.).
- O código deve executar sem erros com "node nome-do-arquivo.mjs".
- Dica de estudo: tente resolver cada exercício antes de olhar o arquivo-modelo, e só consulte depois para conferir e entender as diferenças.

Bloco 1: Algoritmos de Busca

1.1 Busca sequencial com contagem de comparações

Baseado em: 01-busca-sequencial.mjs

Crie uma nova versão da função buscaSequencial que, além de retornar a posição do elemento, também conte e exponha (em uma variável, como foi feito na busca binária com a variável comps) o número de comparações realizadas até encontrar (ou não) o valor buscado.

Utilizando o vetor nomes de data/vetor-nomes.mjs (já ordenado alfabeticamente), busque três nomes: um próximo do início do vetor, um do meio e um do final. Monte uma tabela com o número de comparações de cada busca e explique, em poucas linhas, a relação entre a posição do elemento e o número de comparações (melhor caso, pior caso e caso médio).

1.2 Busca sequencial genérica em vetor de objetos

Baseado em: 02-busca-sequencial-obj.mjs · data/vetor-obj-nomes.mjs (objNomes)

ESTRUTURA DE DADOS II - PROF. ALEXANDRE GOMES

Escreva uma função de comparação (fnComp) que localize a posição do primeiro objeto de objNomes cujo campo classification seja igual a 'F' e cujo frequency_total seja maior que 100000. Teste sua função e informe o first_name do objeto encontrado.

1.3 Busca binária: por que a ordenação é pré-requisito?

Baseado em: 03-busca-binaria.mjs e 04-busca-binaria-obj.mjs

Explique, com suas palavras, por que a busca binária exige que o vetor esteja ordenado. Ilustre com um exemplo concreto: escolha um vetor pequeno desordenado e mostre o que acontece ao aplicar o algoritmo a ele (ele pode não encontrar um valor que existe, ou encontrar uma posição errada).

Em seguida, adapte a função de comparação de 04-busca-binaria-obj.mjs para localizar, em objNomes, o objeto cujo group_name seja igual a um valor de sua escolha. Execute o programa e informe quantas comparações (comps) foram necessárias.

Bloco 2: Recursividade

2.1 Fibonacci: iterativo × recursivo

Baseado em: 09-fatorial-recursividade.mjs

Seguindo o mesmo padrão usado para o fatorial (uma versão iterativa e uma versão recursiva), implemente duas funções que calculem o n-ésimo termo da sequência de Fibonacci (0, 1, 1, 2, 3, 5, 8, 13, ...). Usando console.time/console.timeEnd, meça o tempo de execução das duas versões para n = 10, n = 20 e n = 35.

O que você observa ao comparar os tempos? Por que a versão recursiva de Fibonacci se comporta de forma muito diferente da fatorialRec em termos de desempenho, mesmo as duas sendo recursivas?

2.2 Soma recursiva dos elementos de um vetor

Sem usar: for, while ou reduce()

Implemente uma função recursiva somaVetor(vetor, i = 0) que retorne a soma de todos os elementos numéricos de um vetor. No código, identifique em comentário (como foi feito em fatorialRec) qual é a condição de saída e qual é o caso recursivo.

Bloco 3: Ordenação, Métodos Simples

3.1 Bubble sort: melhor caso, pior caso e caso médio

Baseado em: 05-bubble-sort.mjs e 06-bubble-sort-obj.mjs

Crie uma versão instrumentada do bubble sort (com as variáveis pass, comps e trocas, como em 06-bubble-sort-obj.mjs) e execute-a três vezes sobre um vetor de 10 elementos: (a) já ordenado; (b) ordenado de forma decrescente (pior caso); (c) em ordem aleatória. Monte uma tabela com os valores de pass, comps e trocas de cada execução e comente o resultado obtido.

3.2 Selection sort em dados reais, com ordenação por data

Baseado em: 08-selection-sort-obj.mjs · data/motoristas-obj-desord.mjs (objMotoristas)

Escreva uma função de comparação que ordene objMotoristas por vigencia_do_cadastro, do mais antigo para o mais recente. Atenção: a data está no formato de texto "DD/MM/AAAA", então sua função precisa convertê-la para um formato comparável (por exemplo, "AAAAMMDD") antes de comparar. Ao final, exiba os 5 primeiros e os 5 últimos registros ordenados.

Bloco 4: Ordenação por Divisão e Conquista

4.1 Merge sort com critério de desempate (dois níveis)

Baseado em: 11-merge-sort-obj.mjs · data/motoristas-obj-desord.mjs

Implemente e execute a função de comparação, hoje comentada no arquivo-modelo, que ordena objMotoristas primeiro por `razao_social` e, em caso de empate, por `nome_motorista`. Rode com `console.time` e informe quantas divisões e junções o algoritmo realizou para o tamanho do seu conjunto de dados.

4.2 Quick sort: mudando a escolha do pivô

Baseado em: 12-quick-sort.mjs

O quick sort do arquivo-modelo sempre escolhe o último elemento da região como pivô. Modifique a função para que ela use o primeiro elemento da região (posição ini) como pivô, ajustando a lógica de particionamento conforme necessário.

Depois, faça o dry run (simulação manual, passo a passo, no papel ou em um editor de texto) do vetor [5, 3, 8, 1, 9, 2], mostrando o estado do vetor após cada troca, uma vez com o pivô no fim (versão original) e outra com o pivô no início (sua versão). Guarde essa simulação para revisar antes da prova: é um ótimo jeito de fixar como o particionamento funciona.

Bloco 5: Síntese Comparativa

5.1 Corrida de desempenho entre os quatro algoritmos

Dados: data/nomes-desord.mjs ou data/vetor-obj-nomes.mjs, à sua escolha

Execute bubble sort, selection sort, merge sort e quick sort sobre o mesmo conjunto de dados. Atenção: faça uma cópia do vetor antes de cada ordenação, para não ordenar um vetor que o algoritmo anterior já deixou ordenado.

Para cada algoritmo, registre: tempo de execução (`console.time`), número de comparações e, quando aplicável, número de trocas ou de divisões/junções. Monte uma tabela comparativa e reflita sobre como os resultados obtidos se relacionam com a complexidade teórica (Big-O) de cada algoritmo. Esse é o tipo de raciocínio mais cobrado em prova.

Como usar esta lista para estudar

- Resolva primeiro sem consultar o código-modelo; depois compare sua solução com ele e entenda as diferenças.
- Para os algoritmos de ordenação, tente rodar mentalmente (ou no papel) um exemplo pequeno antes de rodar no computador: é o exercício mais parecido com o que cai em prova.
- Revise, para cada algoritmo, três perguntas: qual é a ideia central, qual é a condição de parada (ou de saída, na recursão) e qual é a complexidade aproximada (Big-O).
- Se sobrar tempo, refaça um exercício de cada bloco sem olhar nada, simulando a prova.